

Guía de buenas prácticas para el desarrollo de aplicaciones web

Asignatura: Ingeniería de Software I

Programa: Ingeniería de Sistemas

Semestre: Quinto

Tecnologías: Spring Boot y Angular

1. Presentación

Durante el desarrollo de una aplicación es común que inicialmente todo funcione correctamente en el computador del desarrollador. Sin embargo, cuando la aplicación pasa a otro ambiente, aparecen situaciones como cambios en las direcciones de las APIs, configuración de bases de datos, puertos, credenciales o servicios externos.

Por esta razón, además de aprender a programar, es necesario conocer algunas prácticas que permitan construir aplicaciones organizadas y fáciles de mantener.

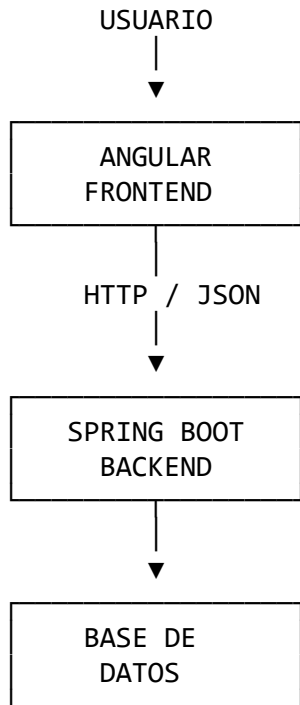
En esta guía se trabajarán cuatro aspectos:

- Separación entre frontend y backend.
- Uso de variables de entorno.
- Manejo de diferentes ambientes de ejecución.
- Consumo de APIs utilizando Spring Boot y Angular.

La idea es que estos conceptos puedan aplicarse directamente en los proyectos desarrollados durante el semestre.

2. Arquitectura propuesta

Para los ejercicios se utilizará una arquitectura sencilla en la que Angular será responsable de la interfaz de usuario y Spring Boot estará encargado de la lógica del servidor.



Esta separación permite que cada aplicación tenga una responsabilidad definida.

Angular

Se encarga principalmente de:

- Mostrar las pantallas.
- Capturar información de formularios.
- Navegar entre las diferentes vistas.
- Consumir las APIs.
- Presentar la información al usuario.

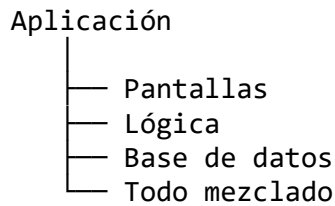
Spring Boot

Se encarga principalmente de:

- Implementar las reglas del negocio.
- Validar información.
- Exponer servicios REST.
- Controlar el acceso a la información.
- Comunicarse con la base de datos.
- Consumir servicios externos cuando sea necesario.

3. ¿Por qué no hacer todo en una sola aplicación?

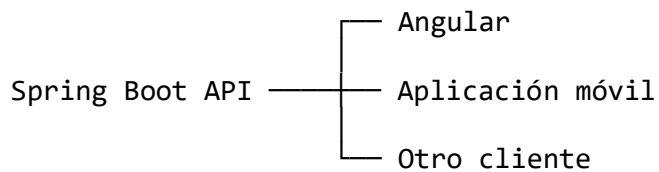
Una aplicación puede comenzar como un proyecto pequeño y funcionar de esta manera:



Cuando el proyecto empieza a crecer, realizar cambios se vuelve más complicado.

Por ejemplo, si posteriormente se necesita desarrollar una aplicación móvil, sería necesario volver a implementar gran parte de la lógica.

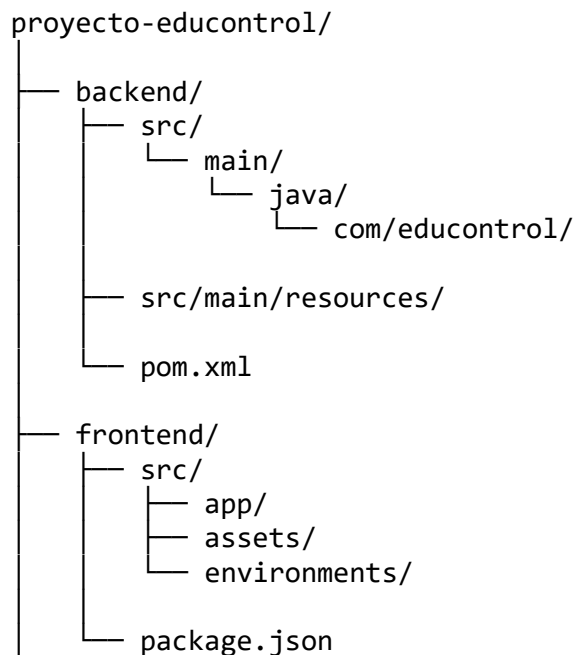
Con una arquitectura desacoplada:



los diferentes clientes pueden utilizar los mismos servicios del backend.

4. Organización del proyecto

Una posible estructura para el proyecto es:



└─ README.md

La idea es que **Angular y Spring Boot sean proyectos independientes**, aunque formen parte del mismo sistema.

5. Ambientes de desarrollo

En un proyecto real no necesariamente existe una única configuración.

Para este ejercicio se trabajarán tres ambientes:

Ambiente	Propósito	Ejemplo
DEV	Desarrollo	Computador del estudiante
PRE	Pruebas	Servidor de pruebas
PROD	Usuarios finales	Servidor productivo

Por ejemplo:

DEV

Angular:
`http://localhost:4200`

Spring Boot:
`http://localhost:8080`

PRE

Angular:
`https://pre.educontrol.com`

Spring Boot:
`https://api-pre.educontrol.com`

PROD

Angular:
`https://educontrol.com`

Spring Boot:
`https://api.educontrol.com`

El problema aparece cuando la URL queda escrita directamente en el código.

Por ejemplo:

```
String url = "http://localhost:8080";
```

Esto funcionaría únicamente en desarrollo.

6. Variables de entorno

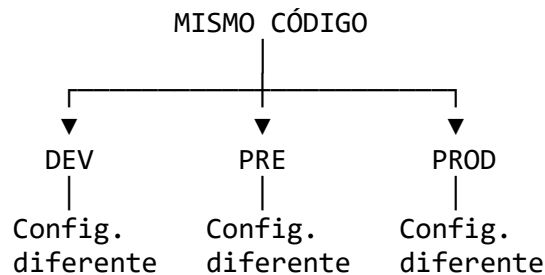
Una variable de entorno permite almacenar un valor que puede cambiar dependiendo del ambiente donde se ejecuta la aplicación.

Por ejemplo:

```
DEV → http://localhost:8080  
PRE → https://api-pre.educontrol.com  
PROD → https://api.educontrol.com
```

El código no debería tener que cambiar cada vez que se mueve la aplicación.

La idea es:



7. Variables de entorno en Spring Boot

Spring Boot permite manejar configuraciones mediante archivos de propiedades o variables de entorno.

Por ejemplo, podemos tener:

```
server.port=${PORT:8080}
```

Esto significa que Spring Boot utilizará la variable PORT.

Si no existe, utilizará 8080.

También podemos definir:

```
spring.datasource.url=${DB_URL}  
spring.datasource.username=${DB_USERNAME}  
spring.datasource.password=${DB_PASSWORD}
```

De esta manera no necesitamos escribir directamente las credenciales dentro del código.

8. Ejemplo de configuración de Spring Boot

Supongamos que tenemos:

```
DB_URL=jdbc:postgresql://localhost:5432/educontrol
DB_USERNAME=educontrol
DB_PASSWORD=123456
```

Nuestro archivo:

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

Spring Boot toma los valores de las variables de entorno.

Importante

No es recomendable escribir esto directamente:

```
spring.datasource.password=123456
```

especialmente cuando el proyecto se encuentra en un repositorio compartido.

9. Perfiles de Spring Boot

Otra alternativa muy utilizada en Spring Boot es trabajar con perfiles.

Por ejemplo:

```
application.properties
application-dev.properties
application-pre.properties
application-prod.properties
```

Desarrollo

```
server.port=8080
```

```
spring.datasource.url=jdbc:postgresql://localhost:5432/educontrol_dev
```

Preproducción

```
server.port=8080
```

```
spring.datasource.url=jdbc:postgresql://servidor-pre:5432/educontrol_pre
```

Producción

```
server.port=8080
```

```
spring.datasource.url=jdbc:postgresql://servidor-  
prod:5432/educontrol_prod
```

Para indicar qué perfil utilizar:

```
spring.profiles.active=dev
```

En ambientes reales este valor también puede ser suministrado como variable de entorno.

10. No guardar credenciales en Git

Uno de los errores que se deben evitar es subir información sensible al repositorio.

Por ejemplo:

```
DB_PASSWORD=MiClave123  
JWT_SECRET=ABC123  
API_KEY=123456789
```

Una alternativa es manejar variables de entorno y utilizar archivos de ejemplo.

Por ejemplo:

```
.env.example
```

con:

```
DB_URL=  
DB_USERNAME=  
DB_PASSWORD=
```

El archivo sirve para indicar qué configuración necesita la aplicación.

Las credenciales reales deben ser suministradas de forma segura al momento de ejecutar la aplicación.

11. Configuración en Angular

En Angular existe una particularidad importante.

Las configuraciones que necesita el frontend pueden manejarse mediante los archivos de environment del proyecto.

Por ejemplo:

```
src/  
└─ environments/  
    └─ environment.ts  
    └─ environment.prod.ts
```

Desarrollo

```
export const environment = {  
  production: false,  
  apiUrl: 'http://localhost:8080'  
};
```

Producción

```
export const environment = {  
  production: true,  
  apiUrl: 'https://api.educontrol.com'  
};
```

Luego podemos utilizar:

```
import { environment } from '../environments/environment';  
  
console.log(environment.apiUrl);
```

12. Una consideración importante sobre Angular

Aquí debemos hacer una aclaración a los estudiantes.

No debemos asumir que una variable configurada en Angular es secreta.

Angular finalmente genera archivos JavaScript que serán enviados al navegador.

Por lo tanto, esto puede estar en Angular:

```
apiUrl: 'https://api.educontrol.com'
```

pero no debemos colocar:

```
databasePassword: '123456'
```

ni:

apiKey: 'CLAVE-SECRETA'

Regla sencilla

Todo lo que llega al navegador puede ser inspeccionado por el usuario.

Las credenciales y secretos deben permanecer en el backend o en mecanismos seguros de gestión de secretos.

13. Creación de una API con Spring Boot

Supongamos que necesitamos consultar estudiantes.

Podemos crear un controlador:

```
@RestController
@RequestMapping("/api/estudiantes")
public class EstudianteController {

    @GetMapping
    public List<Estudiante> listar() {

        return List.of(
            new Estudiante(1L, "Ana"),
            new Estudiante(2L, "Carlos")
        );
    }
}
```

La API podría ser:

GET <http://localhost:8080/api/estudiantes>

Y responder:

```
[
  {
    "id": 1,
    "nombre": "Ana"
  },
  {
    "id": 2,
    "nombre": "Carlos"
  }
]
```

14. Consumo de la API desde Angular

En Angular es recomendable centralizar las llamadas al backend en servicios.

Por ejemplo:

src/app/services/

Creamos:

estudiante.service.ts

Código:

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

import { environment } from '../../environments/environment';

@Injectable({
  providedIn: 'root'
})
export class EstudianteService {

  private apiUrl = environment.apiUrl;

  constructor(private http: HttpClient) {}

  listar(): Observable<any> {

    return this.http.get(
      `${this.apiUrl}/api/estudiantes`
    );
  }
}
```

El componente puede utilizar el servicio:

```
this.estudianteService.listar()
  .subscribe(data => {

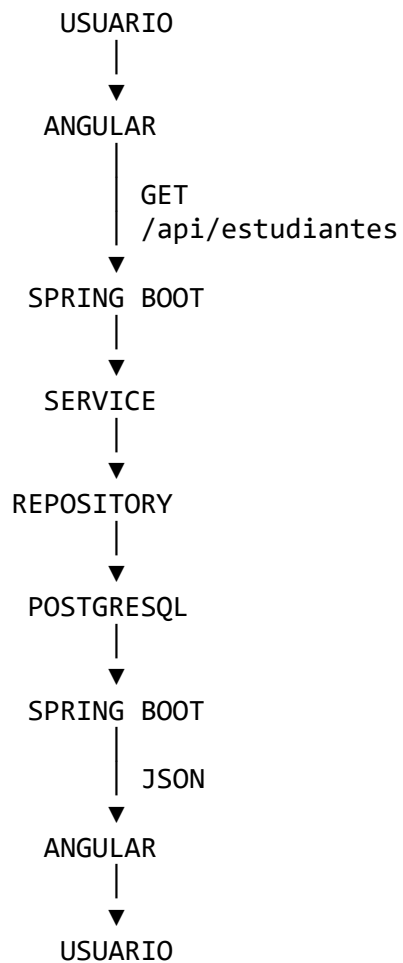
    this.estudiantes = data;

  });
```

De esta manera evitamos colocar llamadas HTTP directamente por todas las pantallas.

15. Flujo completo

Cuando el usuario abre la aplicación:



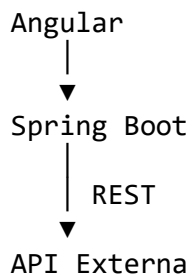
Este flujo permite comprender dónde debe estar cada responsabilidad.

16. ¿Y si Spring Boot necesita consumir otra API?

No siempre el backend solamente consulta la base de datos.

También puede consumir otros servicios.

Por ejemplo:



Supongamos que Spring Boot necesita consultar:

`https://api.externa.com/clientes`

La URL no debería quedar directamente en el código.

Podemos definir:

`cliente.api.url=${CLIENTE_API_URL}`

Y utilizarla desde una clase de configuración o servicio.

Un ejemplo sencillo utilizando `RestClient`:

```
@Service
public class ClienteService {

    private final RestClient restClient;

    public ClienteService(
        @Value("${cliente.api.url}") String apiUrl) {

        this.restClient = RestClient.builder()
            .baseUrl(apiUrl)
            .build();
    }

    public String consultarClientes() {

        return restClient.get()
            .uri("/clientes")
            .retrieve()
            .body(String.class);
    }
}
```

La configuración:

`CLIENTE_API_URL=https://api.externa.com`

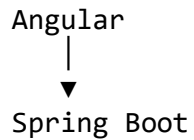
Así, si la URL cambia en PRE o PROD, no tenemos que modificar el código.

17. ¿Quién debería consumir la API?

Hay dos situaciones diferentes.

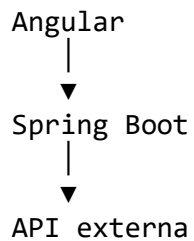
Angular → Spring Boot

Es el escenario principal:



Spring Boot → API externa

Cuando necesitamos integrar otro sistema:



Es importante entender que **Angular no necesariamente debe consumir directamente todos los servicios externos.**

Cuando existen reglas de negocio, autenticación, transformación de datos o información sensible, normalmente es conveniente que la integración sea manejada por el backend.

18. Ejemplo de configuración completa

Desarrollo

Angular

```
export const environment = {  
  production: false,  
  apiUrl: 'http://localhost:8080'  
};
```

Spring Boot

```
DB_URL=jdbc:postgresql://localhost:5432/educontrol_dev  
DB_USERNAME=educontrol  
DB_PASSWORD=*****
```

```
CLIENTE_API_URL=https://api-dev.externa.com
```

Preproducción

Angular

```
export const environment = {
```

```
    production: false,  
    apiUrl: 'https://api-pre.educontrol.com'  
};
```

Spring Boot

```
DB_URL=jdbc:postgresql://servidor-pre:5432/educontrol_pre  
DB_USERNAME=educontrol_pre  
DB_PASSWORD=*****
```

```
CLIENTE_API_URL=https://api-pre.externa.com
```

Producción

Angular

```
export const environment = {  
  production: true,  
  apiUrl: 'https://api.educontrol.com'  
};
```

Spring Boot

```
DB_URL=jdbc:postgresql://servidor-prod:5432/educontrol_prod  
DB_USERNAME=educontrol_prod  
DB_PASSWORD=*****
```

```
CLIENTE_API_URL=https://api.externa.com
```

Lo importante es que **la lógica de la aplicación no cambia.**

Lo que cambia es la configuración.

19. Buenas prácticas que se deben aplicar

Para los proyectos del curso se recomienda tener en cuenta las siguientes reglas:

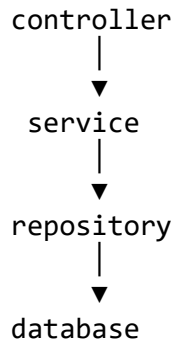
Código

- Utilizar nombres descriptivos.
- Evitar código duplicado.
- Separar responsabilidades.
- Crear métodos pequeños y fáciles de entender.
- Mantener una estructura organizada.

Backend

- Utilizar controladores para recibir las solicitudes.
- Utilizar servicios para implementar reglas de negocio.
- Utilizar repositorios para el acceso a datos.
- Validar la información recibida.
- Manejar adecuadamente los errores.

Una estructura sencilla:



Frontend

- Separar componentes.
- Utilizar servicios para consumir APIs.
- No colocar URLs directamente en los componentes.
- Evitar duplicar llamadas HTTP.
- Separar la lógica de presentación de la lógica de comunicación con el backend.

20. Errores frecuentes

Error 1: colocar la URL directamente

```

this.http.get(
  'http://localhost:8080/api/estudiantes'
);

```

Mejor

```

this.http.get(
  `${environment.apiUrl}/api/estudiantes`
);

```

Error 2: colocar contraseñas en Angular

```

password: '123456'

```

No debe hacerse.

Error 3: guardar credenciales en Git

`spring.datasource.password=123456`

No debe hacerse.

Error 4: acceder directamente desde Angular a la base de datos

Angular $\longrightarrow$ PostgreSQL

No es una arquitectura adecuada.

La comunicación debería ser:

Angular $\rightarrow$ Spring Boot $\rightarrow$ PostgreSQL

Error 5: colocar toda la lógica en el Controller

No es recomendable tener:

```
@GetMapping
public ... {

    // Validaciones
    // reglas de negocio
    // consultas SQL
    // transformación de datos
    // etc.
}
```

Es preferible separar:

```
Controller
  ↓
Service
  ↓
Repository
```

21. Actividad práctica

Para aplicar los conceptos, cada grupo deberá construir una pequeña aplicación denominada:

EduControl - Gestión Académica

La aplicación tendrá:

Frontend - Angular

Pantallas para:

- Listar estudiantes.
- Registrar estudiantes.
- Consultar cursos.
- Registrar cursos.

Backend - Spring Boot

Servicios REST:

GET /api/estudiantes
POST /api/estudiantes

GET /api/cursos
POST /api/cursos

Base de datos

PostgreSQL.

22. Requisitos de la actividad

El proyecto deberá cumplir como mínimo:

- ✓ Angular separado de Spring Boot
- ✓ Spring Boot exponiendo APIs REST
- ✓ Angular consumiendo las APIs
- ✓ Variables de configuración
- ✓ Configuración DEV
- ✓ Configuración PRE
- ✓ Configuración PROD
- ✓ No almacenar contraseñas en el repositorio

✓ Uso de Git

✓ Archivo README.md

✓ Manejo básico de errores

✓ Separación Controller / Service / Repository

23. Preguntas para discusión

Al finalizar la actividad, cada grupo deberá responder:

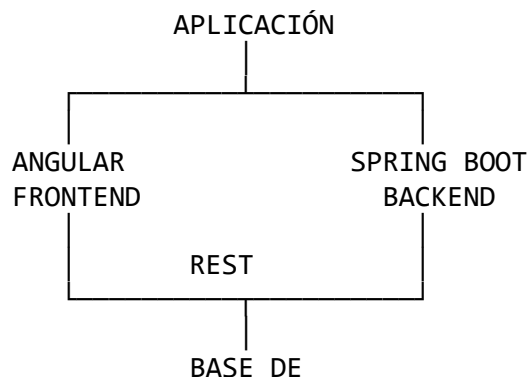
1. ¿Qué problema solucionan las variables de entorno?
2. ¿Por qué una aplicación necesita diferentes configuraciones para DEV, PRE y PROD?
3. ¿Por qué Angular y Spring Boot deberían estar desacoplados?
4. ¿Qué información nunca debería almacenarse en Angular?
5. ¿Por qué no debemos colocar directamente las contraseñas en application.properties?
6. ¿Qué función cumple un Controller?
7. ¿Qué función cumple un Service?
8. ¿Qué función cumple un Repository?
9. ¿Cuál es la diferencia entre consumir una API desde Angular y consumir una API desde Spring Boot?
10. ¿Qué ventajas tiene modificar la configuración sin modificar el código?

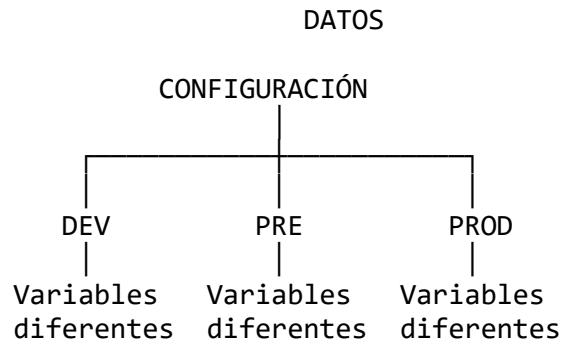
24. Concepto para recordar

Una buena práctica de desarrollo no consiste simplemente en escribir menos código o en utilizar una tecnología determinada.

Consiste en **organizar el software de tal manera que pueda entenderse, mantenerse y modificarse con facilidad.**

Para este ejercicio, la idea principal puede resumirse así:





La aplicación es la misma; lo que cambia entre ambientes es principalmente su configuración.